

Добавление записей в БД с помощью консольной утилиты "tinker"

Создаём основную запись

```
>>> use Illuminate\Support\Facades\Hash;
>>> use App\Models\User;
>>> $user = User::create(['name' => 'admin',
... 'email' => 'admin@bboard.ru',
... 'password' => Hash::make('admin')]);
```

Использование связи "один ко многим"

1-й способ:

```
>>> use App\Models\Bb;
>>> $bb = new Bb();
>>> $bb->title = 'Пылесос';
>>> $bb->content = 'Старый, без шланга';
>>> $bb->price = 500;
>>> $user->bbs()->save($bb);
```

2-й способ:

```
>>> $user->bbs()->create(['title' => 'Грузовик',
... 'content' => 'Грузоподъёмность - 5 т',
... 'price' => 1000000]);
```

3-й способ:

```
>>> $bb = new Bb(['title' => 'Шкаф',
... 'content' => 'Совсем новый, полированный, двухстворчатый',
... 'price' => 1000]);
>>> $bb->user()->associate($user);
>>> $bb->save();
```

Eloquent: Удаление записи в БД. Конспект

// 1)

```
$user = User::find(1);
$user->delete();
```

// 2)

```
User::destroy(1);
User::destroy([1, 2, 3]);
User::destroy(1, 2, 3);
```

// 3)

```
$affectedRows = User::where('votes', '>', 100)->delete();
```

```

/**
 * Создать и вернуть экземпляр несохраненной модели.
 *
 * @return Model
 */
Model::make($attributes = []);

/**
 * Найти модель по ее первичному ключу
 * Можно дополнительно указывать значения каких столбцов необходимы.
 *
 * @return Model | Collection | static[] | static | null
 */
Model::find($id, $columns = ['*']);

/**
 * Найти модель по ее первичному ключу или выбросить исключение
 *
 * @return Model | Collection
 * @throws ModelNotFoundException
 */
Model::findOrFail($id, $columns = ['*']);

/**
 * Найти модели по массиву первичных ключей
 *
 * @return Collection
 */
Model::findMany($ids = [], $columns = ['*']);

/**
 * Найти модель по ее первичному ключу или вернуть новый экземпляр модели.
 *
 * @return Model
 */
Model::findOrCreate($id, $columns = ['*']);

/**
 * Получить первую запись, соответствующую атрибутам или создайте ее.
 *
 * @param array $attributes
 * @param array $values
 *
 * @return Model
 */
Model::firstOrCreate(array $attributes, array $values = []);

/**

```

```

* Создать или обновить запись, соответствующую атрибутам, и заполнить ее значениями.
*
* @param array $attributes
* @param array $values
*
* @return Model
*/

```

```

Model::updateOrCreate(array $attributes, array $values = []);

```

```

/**
* Выполнить запрос и получить первый результат или выбросить исключение.
*
* @param array $columns
*
* @return Model | static
*
* @throws ModelNotFoundException
*/

```

```

Model::firstOrFail($columns = ['*']);

```

```

/**
* Execute the query and get the first result or call a callback.
*
* @param \Closure | array $columns
* @param \Closure | null $callback
*
* @return Model | static | mixed
*/

```

```

Model::firstOr($columns = ['*'], $callback = null);

```

```

/**
* Добавить условие WHERE по первичному ключу к запросу.
*
* @param mixed $id
*
* @return Builder
*/

```

```

Model::whereKey($id);

```

```

/**
* Добавить условие WHERE в запрос.
*
* @param string | array | \Closure $column
* @param string $operator
* @param mixed $value
* @param string $boolean
*
* @return Builder
*/

```

```

Model::where($column, $operator = null, $value = null, $boolean = 'and');

```

```

/**
 * Все доступные операторы условия.
 */
[
    '=', '<', '>', '<=', '>=', '<>', '!=', '<=>',
    'like', 'like binary', 'not like', 'between', 'ilike',
    '&', '|', '^', '<<', '>>',
    'rlike', 'regexp', 'not regexp',
    '~', '~*', '!~', '!~*', 'similar to',
    'not similar to', 'not ilike', '~~*', '!~~*',
];

```

```

/**
 * Добавить условие where() в запрос.
 *
 * @param string | array | \Closure $column
 * @param string $operator
 * @param mixed $value
 *
 * @return Builder | static
 */
Model::orWhere($column, $operator = null, $value = null);

```

```

/**
 * @return Builder
 */
Model::select($columns = ['*']);

```

```

/**
 * Добавить SQL-строку в запрос.
 *
 * @param string $expression
 * @return Builder | static
 */
Model::selectRaw($expression, array $bindings = []);

```

```

/**
 * Добавить SELECT столбца в запрос.
 *
 * @param array|mixed $column
 * @return Builder
 */
Model::addSelect($column);

```

```

/**
 * Добавить подзапрос
 *
 * @param \Closure|Builder|string $query
 * @param string $as
 * @return Builder|static
 */

```

```

* @throws \InvalidArgumentException
*/
Model::selectSub($query, $as);

/**
 * Заставить запрос возвращать только разные результаты.
 *
 * @return Builder
 */
Model::distinct();

/**
 * Добавить таблицу в запрос.
 *
 * @param string $table
 * @return Builder
 */
Model::from($table);

/**
 * Добавить JOIN-запрос
 *
 * @param string $table
 * @param string $first
 * @param string $operator
 * @param string $second
 * @param string $type
 * @param bool $where
 * @return $this
 */
Model::join($table, $first, $operator = null, $second = null, $type = 'inner', $where = false)

/**
 * Получить значение одного столбца из первого результата запроса.
 *
 * @param string $column
 *
 * @return mixed
 */
Model::value($column);

/**
 * Создать коллекцию моделей из простых массивов.
 *
 * @param array $items
 *
 * @return Collection
 */
Model::hydrate($items);

/**

```

```

* Создать коллекцию моделей из необработанного запроса.
*
* @param string $query
* @param array $bindings
*
* @return Collection
*/
Model::fromQuery($query, $bindings = []);

/**
* Выполняет запрос как SELECT.
*
* @param array $columns
*
* @return Collection | static[]
*/
Model::get($columns = ['*']);

/**
* Получить гидратированные модели без активной загрузки.
*
* @param array $columns
*
* @return Model[]
*/
Model::getModels($columns = ['*']);

/**
* Активная (жадная) загрузка отношения для моделей.
*
* @param array $models
*
* @return array
*/
Model::eagerLoadRelations(array $models);

/**
* Получить экземпляр отношения для данного имени отношения.
*
* @param string $name
*
* @return Relation
*/
Model::getRelation($name);

/**
* Разбить результаты запроса, сравнив числовые идентификаторы.
*
* @param string $column
* @param string|null $alias
*
* @return array
*/
Model::groupBy($column, $alias = null);

```

```

* @return bool
*/
Model::chunkById($count, callable $callback, $column = null, $alias = null);

/**
 * Получить коллекцию со значениями данного столбца.
 *
 * @param string      $column
 * @param string|null $key
 *
 * @return Collection
 */
Model::pluck($column, $key = null);

/**
 * Зарегистрируйте новую глобальную область.
 *
 * @param string      $identifier
 * @param Scope | Closure $scope
 *
 * @return Builder
 */
Model::withGlobalScope($identifier, $scope);

/**
 * Разделите заданный запрос (постраничная пагинация).
 *
 * @param int $perPage
 * @param int|null $page
 * @return LengthAwarePaginator
 *
 * @throws \InvalidArgumentException
 */
Model::paginate($perPage = null, $columns = ['*'], $pageName = 'page', $page = null);

/**
 * Простая пагинация.
 *
 * @param int $perPage
 * @param int|null $page
 * @return Paginator
 */
Model::simplePaginate($perPage = null, $columns = ['*'], $pageName = 'page', $page = null);

/**
 * Сохранить новую модель и вернуть экземпляр, созданного объекта.
 *
 * @return Model | Builder
 */
Model::create(array $attributes = []);

```

```
/**
 * Сохранить новую модель и вернуть экземпляр, созданного объекта.
 * Разрешить массовое присвоение.
 *
 * @return Model | Builder
 */
```

```
Model::forceCreate(array $attributes);
```

```
/**
 * Обновить запись в базе данных.
 *
 * @return int
 */
```

```
Model::update(array $values);
```

```
/**
 * Увеличьте значение столбца на заданную величину.
 *
 * @return int
 */
```

```
Model::increment($column, $amount = 1, array $extra = []);
```

```
/**
 * Уменьшить значение столбца на заданную сумму.
 *
 * @return int
 */
```

```
Model::decrement($column, $amount = 1, array $extra = []);
```

```
/**
 * Удаляет запись в БД.
 *
 * @return mixed
 */
```

```
Model::delete();
```

```
/**
 * Полностью удаляет запись, минуя мягкое удаление.
 *
 * @return mixed
 */
```

```
Model::forceDelete();
```

```
/**
 * Зарегистрируйте замену для функции удаления по умолчанию.
 *
 * @param \Closure $callback
 * @return void
 */
```

```
Model::onDelete($callback);
```



```

/**
 * Вызовите данные локальной модели.
 *
 * @param array $scopes
 * @return mixed
 */
Model::scopes(array $scopes);

/**
 * Примените области действия к экземпляру Eloquent builder и верните его.
 *
 * @return Builder|static
 */
Model::applyScopes();

/**
 * Исключает записи, заданной связи
 *
 * @param mixed $relations
 * @return Builder
 */
Model::without($relations);

/**
 * Получить экземпляр базового запроса.
 *
 * @return Builder
 */
Model::getQuery();

/**
 * Задайте базовый экземпляр строителя запроса.
 *
 * @param Builder $query
 * @return Builder
 */
Model::setQuery($query);

/**
 * Получите экземпляр строителя базового запроса.
 *
 * @return Builder
 */
Model::toBase();

/**
 * Создайте новый экземпляр запрашиваемой модели.
 *
 * @return Model
 */
Model::newInstance($attributes = []);

```

```

model::newModelInstance(attributes = []);

/**
 * Получить отношения с нетерпимой загрузкой.
 *
 * @return array
 */
Model::getEagerLoads();

/**
 * Установить отношения, используя нетерпимую загрузку.
 *
 * @param array $eagerLoad
 * @return Builder
 */
public function setEagerLoads(array $eagerLoad);

/**
 * Получить запрос экземпляра модели.
 *
 * @return Model
 */
Model::getModel();

/**
 * Задайте экземпляр модели для запрашиваемой модели.
 *
 * @param Model $model
 * @return Builder
 */
Model::setModel($model);

/**
 * Получите заданный макрос по имени.
 *
 * @param string $name
 * @return \Closure
 */
Model::getMacro($name);

/**
 * Удалить зарегистрированную глобальную область
 *
 * @param Scope | string $scope
 *
 * @return Builder
 */
Model::withoutGlobalScope($scope);

/**
 * Remove all or passed registered global scopes.
 *

```

```
* @return Builder
*/
Model::withoutGlobalScopes(array $scopes = null);

/**
 * Получить массив глобальных областей, которые были удалены из запроса.
 *
 * @return array
 */
Model::removedScopes();
```